

Sorting Quiz Practice Problems

Note 1. In this notes packet, we will study *comparison based* sorting algorithms; that is, sorting algorithms that use the comparison operators $<$, $<=$, $>$, $>=$ to perform the sort. A standard result (which you will not have to prove), is that any comparison based sorting algorithm requires at least $\Omega(n \log n)$ comparison operations. We will see that both merge sort and quick sort are “optimal” in the sense that merge sort has worst case runtime $O(n \log n)$ and quicksort has average case runtime $O(n \log n)$, both of which match the lower bound.

Note 2. The primary use case of sorting is to be able to run binary search later. Recall that binary search takes time $O(\log n)$, but requires sorted data. In order to get this very fast runtime, we must pay a $\Omega(n \log n)$ “setup cost” penalty to sort the data.

Note 3. You should never implement a sorting algorithm in the real world. The built-in python `.sort()` and `sorted()` are much faster than anything you will write for two reasons:

1. They are highly optimized and written in C instead of Python to reduce the constant factor. (Most speed-sensitive functions in python are written in C and python is only a “wrapper” around the C code.)
2. Python uses TimSort. TimSort uses special tricks to reduce the runtime from $O(n \log n)$ to $O(n)$ in certain common situations (like if the data is partially sorted). TimSort was invented by Tim Peters, who is widely considered the second most important person in the python world after Guido.

We are studying sorting algorithms not so that you can write them, but to get you more practice using recursion on real world applications. Also, sorting algorithms form the foundation for more complex algorithms you will learn later.

1 Merge Sort

Note 4. *Merge sort* splits the input into two halves, recursively sorts each half, and merges the two sorted halves into a sorted list. This gives the recurrence

$$T(n) = 2T(n/2) + O(n).$$

By case 2 of the master theorem, $T(n) = O(n \log n)$.

Problem 5. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2
7     return merged(merge_sorted(xs[:mid]), merge_sorted(xs[mid:]))
8
9 def merged(xs, ys):
10    out = []
11    i = 0
12    j = 0
13    while i < len(xs) and j < len(ys):
14        if xs[i] <= ys[j]:
15            out.append(xs[i])
16            i += 1
17        else:
18            out.append(ys[j])
19            j += 1
20    out += xs[i:]
21    out += ys[j:]
22    return out
23
24 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
25 try:
26     zs = merge_sorted(xs)
```

Problem 6. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2 + 1
7     return merged(merge_sorted(xs[:mid]), merge_sorted(xs[mid:]))
8
9 def merged(xs, ys):
10     out = []
11     i = 0
12     j = 0
13     while i < len(xs) and j < len(ys):
14         if xs[i] <= ys[j]:
15             out.append(xs[i])
16             i += 1
17         else:
18             out.append(ys[j])
19             j += 1
20     out += xs[i:]
21     out += ys[j:]
22     return out
23
24 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
25 try:
26     zs = merge_sorted(xs)
27     print('zs=', zs)
28 except RuntimeError:
29     print('StackOverflow')
30 EOF
31 $ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

Problem 7. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2
7     return merged(merge_sorted(xs[:mid]), merge_sorted(xs[mid + 1:]))
8
9 def merged(xs, ys):
10     out = []
11     i = 0
12     j = 0
13     while i < len(xs) and j < len(ys):
14         if xs[i] <= ys[j]:
15             out.append(xs[i])
16             i += 1
17         else:
18             out.append(ys[j])
19             j += 1
20     out += xs[i:]
21     out += ys[j:]
22     return out
23
24 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
25 try:
26     zs = merge_sorted(xs)
27     print('zs=', zs)
28 except RuntimeError:
29     print('StackOverflow')
30 EOF
31 $ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 8. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted(xs):
4     mid = len(xs) // 2
5     return merged(merge_sorted(xs[:mid]), merge_sorted(xs[mid:]))
6
7 def merged(xs, ys):
8     out = []
9     i = 0
10    j = 0
11    while i < len(xs) and j < len(ys):
12        if xs[i] <= ys[j]:
13            out.append(xs[i])
14            i += 1
15        else:
16            out.append(ys[j])
17            j += 1
18    out += xs[i:]
19    out += ys[j:]
20    return out
21
22 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
23 try:
24     zs = merge_sorted(xs)
25     print('zs=', zs)
26 except RuntimeError:
27     print('StackOverflow')
28 EOF
29 $ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 9. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2
7     return merged(merge_sorted(xs[:mid]), merge_sorted(xs[mid:]))
8
9 def merged(xs, ys):
10     out = []
11     i = 0
12     j = 0
13     while i < len(xs) and j < len(ys):
14         if xs[i] <= ys[j]:
15             out.append(xs[i])
16             i += 1
17         else:
18             out.append(ys[j])
19             j += 1
20     return out
21
22 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
23 try:
24     zs = merge_sorted(xs)
25     print('zs=', zs)
26 except RuntimeError:
27     print('StackOverflow')
28 EOF
29 $ python3 foo.py
```

Fraction of LLMs with correct answer: 8 / 16 = 0.50

Problem 10. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2
7     return merged(merge_sorted(xs[:mid]), merge_sorted(xs[mid:]))
8
9 def merged(xs, ys):
10     out = []
11     i = 0
12     j = 0
13     while i < len(xs) and j < len(ys):
14         if xs[i] >= ys[j]:
15             out.append(xs[i])
16             i += 1
17         else:
18             out.append(ys[j])
19             j += 1
20     out += xs[i:]
21     out += ys[j:]
22     return out
23
24 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
25 try:
26     zs = merge_sorted(xs)
27     print('zs=', zs)
28 except RuntimeError:
29     print('StackOverflow')
30 EOF
31 $ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 11. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted(xs):
4     def go(lo, hi):
5         if hi - lo <= 1:
6             return xs[lo:hi]
7         mid = (lo + hi) // 2
8         return merged(go(lo, mid), go(mid, hi))
9     return go(0, len(xs))
10
11 def merged(xs, ys):
12     out = []
13     i = 0
14     j = 0
15     while i < len(xs) and j < len(ys):
16         if xs[i] <= ys[j]:
17             out.append(xs[i])
18             i += 1
19         else:
20             out.append(ys[j])
21             j += 1
22     out += xs[i:]
23     out += ys[j:]
24     return out
25
26 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
27 try:
28     zs = merge_sorted(xs)
29     print('zs=', zs)
30 except RuntimeError:
31     print('StackOverflow')
32 EOF
33 $ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Note 12. *3-way merge sort* is a variation of merge sort that splits the input into three (roughly) equal pieces, recursively sorts each piece, and then merges all three sorted lists together. The recurrence is

$$T(n) = 3T(n/3) + O(n).$$

Even though we are making more recursive calls, the master theorem still gives $T(n) = O(n \log n)$.

Problem 13. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted3(xs):
4     if len(xs) <= 1:
5         return xs
6     mid1 = len(xs) // 3
7     mid2 = 2 * len(xs) // 3
8     return merged(
9         merged(merge_sorted3(xs[:mid1]),
10              merge_sorted3(xs[mid1:mid2])),
11         merge_sorted3(xs[mid2:]))
12
13 def merged(xs, ys):
14     out = []
15     i = 0
16     j = 0
17     while i < len(xs) and j < len(ys):
18         if xs[i] <= ys[j]:
19             out.append(xs[i])
20             i += 1
21         else:
22             out.append(ys[j])
23             j += 1
24     out += xs[i:]
25     out += ys[j:]
26     return out
27
28 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
29 try:
30     zs = merge_sorted3(xs)
31     print('zs=', zs)
32 except RuntimeError:
33     print('StackOverflow')
34 EOF
35 $ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 14. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted3(xs):
4     if len(xs) <= 1:
5         return xs
6     mid1 = len(xs) // 3
7     mid2 = 2 * len(xs) // 3
8     return merged(
9         merged(merge_sorted3(xs[:mid1]),
10             merge_sorted3(xs[mid1 + 1:mid2])),
11         merge_sorted3(xs[mid2:]))
12
13 def merged(xs, ys):
14     out = []
15     i = 0
16     j = 0
17     while i < len(xs) and j < len(ys):
18         if xs[i] <= ys[j]:
19             out.append(xs[i])
20             i += 1
21         else:
22             out.append(ys[j])
23             j += 1
24     out += xs[i:]
25     out += ys[j:]
26     return out
27
28 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
29 try:
30     zs = merge_sorted3(xs)
31     print('zs=', zs)
32 except RuntimeError:
33     print('StackOverflow')
34 EOF
35 $ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

Problem 15. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def merge_sorted3(xs):
4     if len(xs) == 0:
5         return xs
6     mid1 = len(xs) // 3
7     mid2 = 2 * len(xs) // 3
8     return merged(
9         merged(merge_sorted3(xs[:mid1]),
10              merge_sorted3(xs[mid1:mid2])),
11         merge_sorted3(xs[mid2:]))
12
13 def merged(xs, ys):
14     out = []
15     i = 0
16     j = 0
17     while i < len(xs) and j < len(ys):
18         if xs[i] <= ys[j]:
19             out.append(xs[i])
20             i += 1
21         else:
22             out.append(ys[j])
23             j += 1
24     out += xs[i:]
25     out += ys[j:]
26     return out
27
28 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
29 try:
30     zs = merge_sorted3(xs)
31     print('zs=', zs)
32 except RuntimeError:
33     print('StackOverflow')
34 EOF
35 $ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

Note 16. Quicksort is famous for having an $O(n^2)$ worst case runtime, but also usually being faster than merge sort. This is for two reasons:

1. The worst case runtime rarely happens. When the input list is randomly arranged, quicksort has average runtime $O(n \log n)$ which matches merge sort.
2. quicksort has a small *constant factor*.

Average case runtime analysis requires tracking probabilities and is beyond the scope of this class.

Problem 17. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2
7     pivot = xs[mid]
8     smaller = [x for x in xs if x < pivot]
9     equal    = [x for x in xs if x == pivot]
10    larger   = [x for x in xs if x > pivot]
11    return quick_sorted(smaller) + equal + quick_sorted(larger)
12
13 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
14 try:
15     zs = quick_sorted(xs)
16     print('zs=', zs)
17 except RuntimeError:
18     print('StackOverflow')
19 EOF
20 $ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 18. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     pivot = xs[0]
7     smaller = [x for x in xs if x <= pivot]
8     larger = [x for x in xs if x > pivot]
9     return quick_sorted(smaller) + quick_sorted(larger)
10
11 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
12 try:
13     zs = quick_sorted(xs)
14     print('zs=', zs)
15 except RuntimeError:
16     print('StackOverflow')
17 EOF
18 $ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 19. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2
7     pivot = xs[mid]
8     smaller = [x for x in xs if x <= pivot]
9     larger = [x for x in xs if x > pivot]
10    return quick_sorted(smaller) + quick_sorted(larger)
11
12 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
13 try:
14     zs = quick_sorted(xs)
15     print('zs=', zs)
16 except RuntimeError:
17     print('StackOverflow')
18 EOF
19 $ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 20. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     pivot = xs[-1]
7     smaller = [x for x in xs if x <= pivot]
8     larger = [x for x in xs if x > pivot]
9     return quick_sorted(smaller) + quick_sorted(larger)
10
11 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
12 try:
13     zs = quick_sorted(xs)
14     print('zs=', zs)
15 except RuntimeError:
16     print('StackOverflow')
17 EOF
18 $ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 21. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_sorted(xs):
4     if len(xs) <= 1:
5         return xs
6     mid = len(xs) // 2
7     pivot = xs[mid]
8     smaller = [x for x in xs if x < pivot]
9     larger = [x for x in xs if x >= pivot]
10    return quick_sorted(smaller) + quick_sorted(larger)
11
12 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
13 try:
14     zs = quick_sorted(xs)
15     print('zs=', zs)
16 except RuntimeError:
17     print('StackOverflow')
18 EOF
19 $ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Note 22. *Quick select* is a variant of quick sort that returns only the k -th smallest element of the input, indexed from 0. It is commonly used in data science applications when trying to calculate the median or percentile of a dataset.

Because we only need to recurse into the partition that contains the k -th element, the recurrence is

$$T(n) = T(n/2) + O(n),$$

which gives $O(n)$ time in the average case despite quick select sharing quick sort's $O(n^2)$ worst case.

Problem 23. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_select(xs, k):
4     if len(xs) == 1:
5         return xs[0]
6     mid = len(xs) // 2
7     pivot = xs[mid]
8     smaller = [x for x in xs if x < pivot]
9     equal    = [x for x in xs if x == pivot]
10    larger   = [x for x in xs if x > pivot]
11    if k < len(smaller):
12        return quick_select(smaller, k)
13    if k < len(smaller) + len(equal):
14        return pivot
15    return quick_select(larger, k - len(smaller) - len(equal))
16
17 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
18 try:
19     zs = quick_select(xs, 5)
20     print('zs=', zs)
21 except RuntimeError:
22     print('StackOverflow')
23 EOF
24 $ python3 foo.py
```

Fraction of LLMs with correct answer: $15 / 16 = 0.94$

Problem 24. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_select(xs, k):
4     if len(xs) == 1:
5         return xs[0]
6     mid = len(xs) // 2
7     pivot = xs[mid]
8     smaller = [x for x in xs if x < pivot]
9     equal    = [x for x in xs if x == pivot]
10    larger   = [x for x in xs if x > pivot]
11    if k < len(smaller):
12        return quick_select(smaller, k)
13    if k < len(smaller) + len(equal):
14        return pivot
15    return quick_select(larger, k - len(smaller) - len(equal))
16
17 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
18 try:
19     zs = quick_select(xs, 0)
20     print('zs=', zs)
21 except RuntimeError:
22     print('StackOverflow')
23 EOF
24 $ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 25. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 def quick_select(xs, k):
4     if len(xs) == 1:
5         return xs[0]
6     mid = len(xs) // 2
7     pivot = xs[mid]
8     smaller = [x for x in xs if x <= pivot]
9     larger  = [x for x in xs if x > pivot]
10    if k < len(smaller):
11        return quick_select(smaller, k)
12    return quick_select(larger, k - len(smaller))
13
14 xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
15 try:
16     zs = quick_select(xs, 5)
17     print('zs=', zs)
18 except RuntimeError:
19     print('StackOverflow')
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

2 Built-in Sorting

Note 26. In python, both `sorted(xs)` and `xs.sort()` accept a `key` argument, which is a function that determines the sort order.

Note 27. Example key functions include the builtins `len` and `abs`, the string methods `str.lower` and `str.upper`, and functions from the `operator` module such as `operator.itemgetter` (which fetches an index) and `operator.attrgetter` (which fetches an attribute). The `lambda` keyword creates an anonymous function in-line, which is convenient for small expressions like `lambda d: d['age']`. All of these are ordinary functions, so they can be passed around, stored in variables, or returned from other functions.

Note 28. The key function is called *once* per element, so a sort using `key=` makes $\Theta(n)$ calls to the key function, whereas the older style of passing a comparison function via `functools.cmp_to_key` makes $O(n \log n)$ calls. Both produce the same sorted order, but the key style is much faster in practice because the expensive python-level call happens outside the inner sorting loop.

Problem 29. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 xs = [-3, 7, -1, 8, -5, 10, -2, 9, -4, 6]
4 zs = sorted(xs, key=abs)
5 print('zs=', zs)
6 EOF
7 $ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 30. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 xs = ['banana', 'kiwi', 'apple', 'fig', 'cherry', 'date']
4 zs = sorted(xs, key=len)
5 print('zs=', zs)
6 EOF
7 $ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 31. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 people = [
4     {'name': 'carol', 'age': 40},
5     {'name': 'alice', 'age': 30},
6     {'name': 'bob', 'age': 20},
7 ]
8 zs = sorted(people, key=lambda d: d['name'])
9 print('zs=', zs)
10 EOF
11 $ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 32. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 people = [
4     {'name': 'carol', 'age': 40},
5     {'name': 'alice', 'age': 30},
6     {'name': 'bob', 'age': 20},
7 ]
8 zs = sorted(people, key=lambda d: d['age'])
9 print('zs=', zs)
10 EOF
11 $ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 33. Write the output of the final command in the following shell script.

```
1 $ cd; rm -rf quiz; mkdir quiz; cd quiz
2 $ cat > foo.py <<EOF
3 import functools
4 def cmp(a, b):
5     if len(a) != len(b):
6         return len(a) - len(b)
7     if a < b:
8         return -1
9     return 1 if a > b else 0
10 xs = ['banana', 'kiwi', 'apple', 'fig', 'cherry', 'date']
11 zs = sorted(xs, key=functools.cmp_to_key(cmp))
12 print('zs=', zs)
13 EOF
14 $ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

LLM Model Performance

